

Controle de Versão com Git

*O mesmo terminal da semana passada — agora
guardando o histórico do seu código*



```
$ cd projetos
$ git init
Initialized empty Git repository
$ git status
On branch main
$ _
```



Prof. Dr. Domingos Bernardo Gomes Santos
Instituto Federal de São Paulo — Curso Técnico



Por Que Versionar?

trabalho_final_V2_FINAL_agora_vai.docx

Você já deve ter salvo um arquivo assim. É esse problema que o Git resolve — de um jeito automático e organizado.



Você já conhece

Renomear arquivos manualmente para guardar versões



Vamos aprender

Git: histórico automático, sem precisar renomear nada



```
$ ls
trabalho.docx
trabalho_v2.docx
trabalho_final.docx
trabalho_final_v2.docx
trabalho_FINAL_agora_vai.docx

# com Git, é só:
$ git commit -m "versão 2"
$ git commit -m "versão final"
```



Centralizado vs. Distribuído

6.3.1 — os dois modelos de controle de versão

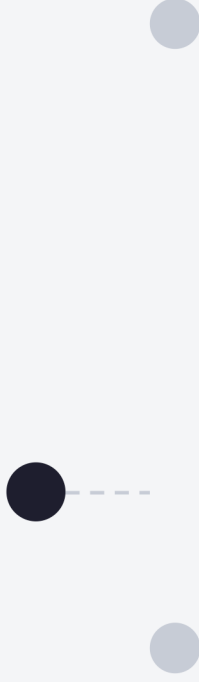


Centralizado

Um único servidor central guarda todo o histórico.

Sem internet, ninguém trabalha.

Exemplos: SVN, CVS



Distribuído (Git)

Cada pessoa tem uma cópia completa do histórico, na própria máquina.

Funciona offline. Sincroniza quando quiser.

É o modelo do Git





O Que É o Git

6.3.2 — o programa que faz o controle de versão distribuído

Git é um programa que roda no seu computador e **tira fotografias (snapshots)** do estado dos seus arquivos, sempre que você pede.

Cada fotografia vira um ponto no histórico do projeto — e você pode voltar a qualquer uma delas quando quiser.



Git ≠ GitHub

Git é a ferramenta. GitHub é um site que guarda repositórios Git na nuvem — vemos isso numa próxima aula.



```
$ git --version  
git version 2.43.0
```



Relembrando o Terminal

O Git não é um programa separado — ele roda com os mesmos comandos da aula passada.

pwd

onde você está

ls

o que tem na pasta

cd

navegar entre pastas

mkdir

criar uma pasta

touch

criar um arquivo

cat

ver o conteúdo

Hoje, esses comandos vão preparar o terreno antes de cada comando novo do Git.



Configurando Sua Identidade

Antes do primeiro commit, o Git precisa saber quem é você
— isso só se faz uma vez por computador.

--global

vale para todos os projetos nesse computador

user.name

o nome que vai aparecer nos seus commits

user.email

o e-mail associado aos seus commits

```
$ git config --global \
  user.name "Ana Silva"
$ git config --global \
  user.email "ana@aluno.ifsp.edu.br"

$ git config --global --list
user.name=Ana Silva
user.email=ana@aluno.ifsp.edu.br
```



Criando o Repositório

Três passos: navegar até a pasta (terminal), criar a pasta se precisar, e transformar em repositório Git.

1

mkdir projeto-git

cria a pasta do projeto — comando da semana passada

2

cd projeto-git

entra na pasta — comando da semana passada

3

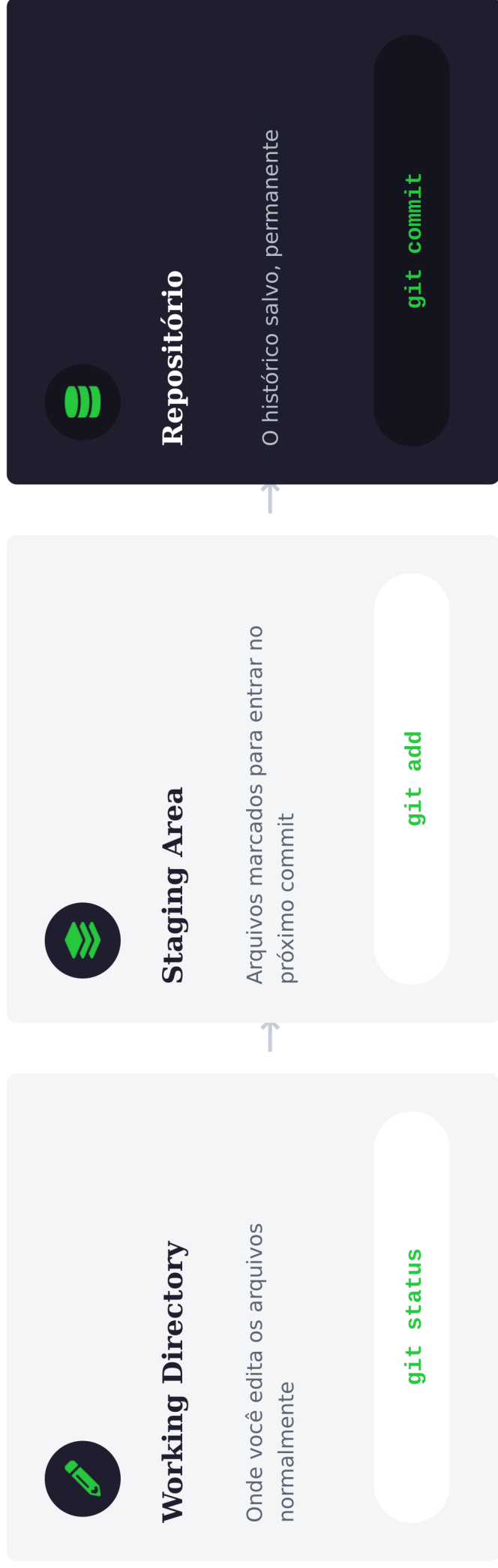
git init

transforma a pasta em um repositório Git

```
$ mkdir projeto-git
$ cd projeto-git
$ git init
Initialized empty Git repository in
.../projeto-git/.git/
$ ls -a
. .. .git
```



O Ciclo Básico do Git





Seu Primeiro Commit

Do arquivo em branco até o histórico salvo — passo a passo, combinando terminal e Git.

touch README.md

cria o arquivo (terminal)

git status

mostra o arquivo como "não rastreado"

git add README.md

marca o arquivo para o commit

git commit -m "..."

salva o snapshot no histórico

```
$ touch README.md
$ git status
Untracked files:
  README.md
$ git add README.md
$ git commit -m "primeiro commit"
[main (root-commit) a1b2c3d]
primeiro commit
1 file changed, 0 insertions(+)
```



Vendo o Histórico

git log

Mostra todos os commits já feitos: quem, quando e a mensagem de cada um.



Cada commit é permanente

Diferente do rm — nada se perde. Você sempre pode voltar no histórico.

```
● ● ●
$ git log
commit a1b2c3d4e5f6...
Author: Ana Silva <ana@...>
Date:   Tue Aug 11 14:32:00 2026

    primeiro commit
```



Seu Cheat Sheet de Git

git init

Transforma a pasta atual em repositório

git config --global

Configura seu nome/e-mail (uma vez só)

git status

Mostra o que mudou desde o último commit

git add <arquivo>

Marca um arquivo para o próximo commit

git commit -m "..."

Salva um snapshot com uma mensagem

git log

Mostra o histórico de commits

Lembrete: mkdir, cd, ls e touch continuam valendo — Git só entra depois deles.



Exercícios no Moodle



Os exercícios práticos desta aula estão no Moodle.

Acesse a página da disciplina SPOIFDS e pratique os comandos que vimos hoje.

O que praticar

Configuração

```
git config --global
```

Repositório

```
git init, git status
```

Primeiro commit

```
git add, git commit, git log
```

Dúvidas? Revise os slides desta aula ou traga para a próxima.



Antes de Continuar

Tudo o que vimos hoje é local.

Ninguém mais vê seus commits até você decidir compartilhar — isso vem na próxima aula.



Próxima aula: branches e GitHub

Esses mesmos comandos vão aparecer de novo — agora para trabalhar em equipe.



```
$ git status
On branch main
$ _ (te vejo na próxima aula)
```



Obrigado pela Atenção!

Continue praticando — te vejo na próxima aula, agora com branches e GitHub.

Prof. Dr. Domingos Bernardo Gomes Santos
SPOIFDS · Instituto Federal de São Paulo